Map-STDP: Does it form cortices?

Vikram Ramavarapu

Abstract

STDP is a local learning rule that may not capture the global modular structure of the neural network. In this paper, a new Map-STDP rule is derived using the map equation, which is an info-theoretic objective function for community detection.

1 Introduction

1.1 Problem

Spike-timing dependent plasticity (STDP) [1] is a biologically plausible learning rule that has been widely used in spiking neural networks. When a pre-synaptic neuron fires shortly before a post-synaptic neuron, the synapse between them is strengthened. Conversely, if the pre-synaptic neuron fires shortly after the post-synaptic neuron, the synapse is weakened.

However, STDP is a fully local rule that may not capture the global structure of the neural network. Ideally, we want our learning rule to guide our network to form *cortices*, which are densely connected communities of neurons that are sparsely connected to other communities.

1.2 Why Cortices?

Cortices enable expressive spatial population codes that can perform representations such that cortices can be mapped to concepts.

For example, in the case of ImageNet classification, we want the network to learn a modular structure where each community corresponds to a different class. This would allow the network to learn a more efficient representation of the data, as it would be able to leverage the modular structure to learn more compact representations of each class.

In the case of the Gymnasium car racing environment, we would want the network to form five cortices that correspond to: do nothing, accelerate, brake, turn left, and turn right.

1.3 Random Walks and the Map Equation

Neural dynamics are very stochastic and operate as a form of random walk [2], which is a major component of the map equation, an info-theoretic objective function for community detection in the Infomap algorithm [3]. The map equation measures the modular structure of a network by quantifying the amount of information needed to describe the flow of activity through the network via a random walk.

As such, a Map-STDP rule that incorporates the map equation is derived. The Map-STDP rule is a local learning rule that can capture the global modular structure of the neural network. Moreover, the Map-STDP rule preserves biorealism through the use of info-theoretic entropy maximization, stochastic neural dynamics, and local synaptic updates.

2 Infomap and Entropy

2.1 Infomap

The Infomap algorithm is a community detection algorithm that minimizes the map equation to identify communities in a network. The map equation is defined as:

$$L(M) = q_{\curvearrowright} H(\mathcal{Q}) + \sum_{m \in M} p^m_{\circlearrowright} H(\mathcal{P}^m)$$

where M is a partition of the network into m communities. There are two terms. The first term represents the entropy of movement between communities, will the second term represents the entropy of movement within communities, where exiting the community is also considered a movement.

2.2 Connecting the Map Equation to Neural Computation

Map-STDP is designed to be compounded with a standard STDP rule that maximizes mutual information between stimulus and neural response, a well-established consequence of Hebbian learning under information-theoretic objectives [4]. The STDP term handles single-neuron expressivity, while the map equation term acts as a structural constraint at the population level.

The deeper grounding comes from variational free energy [5]. The SNN's MCMC dynamics [2] produce a stationary distribution $p(\mathbf{z}|\mathbf{o})$ over neural activity patterns given sensory input $\mathbf{o}$. This is precisely the recognition density that minimizes variational free energy. This distribution represents the network's implicit model of the world. The map equation operates on the marginal visit frequencies $\tilde{p}_i = p(z_i = 1|\mathbf{o})$, which are byproducts of this same sampling process. Map-STDP therefore does not introduce a separate objective: it imposes community structure on the distribution the network is already converging to, with the addition of mesoscale organization of $p(\mathbf{z}|\mathbf{o})$.

3 Deriving Map-STDP

As mentioned earlier, in order to properly address response entropy, the Map-STDP rule should be compounded with an STDP rule that maximizes response entropy. As such, we can start with a rule in the form of:

$$\Delta W_{ij} = \eta \cdot \text{STDP}(i,j) - \eta' \cdot G(i,j) \tag{1}$$

where $\text{STDP}(i,j)$ is a standard STDP rule that maximizes response entropy, $G(i,j)$ is a modular gating function, and η, η' are separate learning rates for the two objectives. The two terms are additive rather than multiplicative, so each objective acts independently: STDP drives the synaptic updates toward maximizing response entropy, while the map gradient term descends the map equation loss to encourage modular structure. The goal is to derive the form of $G(i,j)$.

In its general form, a random walk on a network has a transition matrix T that can be defined as:

$$T_{ij} = \frac{W_{ij}}{\sum_k W_{ik}}$$

where W_{ij} is the weight of the edge from node i to node j. We can then tie this in to Buesing et. al.'s equation for the membrane potential of a neuron in an SNN that performs MCMC sampling [2]:

$$u_i = b_i + \sum_j W_{ij} z_j(t)$$

where u_i is the membrane potential of neuron i, b_i is the bias of neuron i, and $z_j(t)$ is the binary spike variable of neuron j at time t. While z is an instantaneous variable, we can take its expectation over time in order to make it compatible with the map equation's flow-based framework. This gives us:

$$\langle z_j \rangle = \tilde{p}_j = p(z_j = 1 | \mathbf{o})$$

This gives us the corrected definition of our transition matrix as:

$$\tilde{T}_{ij} = \frac{W_{ij}\tilde{p}_j}{\sum_k W_{ik}\tilde{p}_k} = \frac{\tilde{W}_{ij}}{\tilde{s}_i}$$

With our new defined variables $\tilde{W}_{ij} = W_{ij}\tilde{p}_j$ and $\tilde{s}_i = \sum_k W_{ik}\tilde{p}_k$.

The Chapman-Kolmogorov equation then gives us the probability of being at node j at time t as:

$$\pi_j^{t+1} = \sum_i \tilde{T}_{ij}\pi_i^t$$

The stationary distribution therefore satisfies:

$$\tilde{p}_j = \sum_i \tilde{T}_{ij}\tilde{p}_i \rightarrow \tilde{p} = \tilde{T}\tilde{p}$$

We can rule out the trivial solution of $\tilde{p} = 0$ by enforcing a normalization constraint of $\sum_j \tilde{p}_j = 1$. Another trivial solution we can rule out is the one where $\tilde{T} = I$, which could only be true if we don't add a constraint of irreducibility, meaning there must be some level of connectivity between all nodes in the network. This is then reducible to a fixed point problem, therefore we can then solve for $\tilde{p}$ using power iteration:

$$\tilde{p}^{(k+1)} = \tilde{T}\tilde{p}^{(k)}$$

Which can be estimated online via the local update rule:

$$\tilde{p}_i \leftarrow (1-\beta)\tilde{p}_i + \beta \sum_j \tilde{T}_{ij} z_j(t) \tag{2}$$

It is worth noting that the stationary probability $\tilde{p}$ is also the PageRank of the network with transition matrix $\tilde{T}$. The above rule is actually a form of the iterative step of the power iteration method for computing PageRank.

The proofs of convergence are deferred to Appendix A.

Now, it's time to come back to the map equation and derive the form of $G(i,j)$. We can rewrite $p^m_{\circlearrowleft}$ and $q_{\curvearrowright}$ in terms of $\tilde{p}$:

$$\begin{aligned} p_m &:= \sum_{i \in m} \tilde{p}_i \\ q_m &:= \sum_{i \in m} \sum_{j \notin m} \tilde{T}_{ij} \tilde{p}_i \\ p^m_{\circlearrowleft} &= p_m + q_m \\ q_{\curvearrowright} &= \sum_{m \in M} q_m \end{aligned}$$

Where p_m is the total visit frequency of community m, and q_m is the total exit frequency of community m. We can then rewrite the map equation as:

$$L(M) = q_{\curvearrowright} H(\mathcal{Q}) + \sum_{m \in M} (p_m + q_m) H(\mathcal{P}^m)$$

Now we need to expand the entropy terms.

$$H(\mathcal{Q}) = -\sum_{m \in M} \frac{q_m}{q_{\curvearrowright}} \log\left(\frac{q_m}{q_{\curvearrowright}}\right)$$

$$H(\mathcal{P}^m) = -\frac{q_m}{p_m + q_m} \log\left(\frac{q_m}{p_m + q_m}\right) - \sum_{i \in m} \frac{\tilde{p}_i}{p_m + q_m} \log\left(\frac{\tilde{p}_i}{p_m + q_m}\right)$$

Now, substituting this back into the map equation gives us:

$$L(M) = -\sum_{m \in M} q_m \log\left(\frac{q_m}{q_{\curvearrowright}}\right) + \sum_{m \in M} q_m \log\left(\frac{q_m}{p_m + q_m}\right) + \sum_{m \in M} \sum_{i \in m} \tilde{p}_i \log\left(\frac{\tilde{p}_i}{p_m + q_m}\right)$$

Now we can take the gradient with respect to W_{ij}. The first term requires two dependent derivatives via the chain rule: $\partial q_m/\partial W_{ij}$ and $\partial q_\curvearrowright/\partial W_{ij}$. The second term also requires two dependent derivatives: $\partial q_m/\partial W_{ij}$ and $\partial p_m/\partial W_{ij}$. The third term requires two dependent derivatives: $\partial \tilde{p}_i/\partial W_{ij}$ and $\partial p_m/\partial W_{ij}$.

When computing $\partial \tilde{p}_i/\partial W_{ij}$:

$$\frac{\partial \tilde{p}_i}{\partial W_{ij}} = \left[\frac{\partial \tilde{T}\tilde{p}}{\partial W_{ij}}\right]_i$$
$$= \left[\frac{\partial \tilde{T}}{\partial W_{ij}}\tilde{p} + \tilde{T}\frac{\partial \tilde{p}}{\partial W_{ij}}\right]_i = \left[\frac{\partial \tilde{p}}{\partial W_{ij}}\right]_i$$

Removing the $[\cdot]_i$ operator gives us a fixed point partial differential equation. We can solve this as follows:

$$\frac{\partial \tilde{p}}{\partial W_{ij}} = (I - \tilde{T})^{-1}\frac{\partial \tilde{T}}{\partial W_{ij}}\tilde{p}$$

This raises the issue of a global matrix inversion. We sidestep it by treating $\tilde{p}$ as constant with respect to W_{ij} during the synaptic update, consistent with an EM interpretation: the E-step re-estimates $\tilde{p}$ via Equation 2, while the M-step optimizes W with $\tilde{p}$ held fixed. Under this approximation, $\partial p_m/\partial W_{ij} = 0$.

When we derive $G(i,j)$ under this approximation, we get the following form:

$$G(i,j) = \tilde{p}_i\tilde{p}_j\frac{\mathbb{I}[j \notin m(i)]\tilde{s}_i - e_i}{\tilde{s}_i^2}\log\frac{q_\curvearrowright}{p_{m(i)} + q_{m(i)}}$$

The full derivation can be found in Appendix A.

Where $e_i = \sum_{j' \notin m(i)} W_{ij'}\tilde{p}_{j'}$ is the total exit flow from node i. The gating function $G(i,j)$ is the same as the gradient of the map equation with respect to W_{ij}, treating $\tilde{p}$ as constant with respect to W.

We can potentially simplify this equation by removing terms that have constant sign, assuming they get absorbed into the learning rate. For example, $\tilde{p}_i$, $\tilde{p}_j$, and $\tilde{s}_i$ are always positive, so they can be removed without affecting the direction of the update. We want to keep our network modular. This means that so long as we keep our learning rate negative (which in Equation 1, we do), we can remove the log term as well, since $q_\curvearrowright < p_{m(i)} + q_{m(i)}$ is a necessary condition for modularity. This gives us the following simplified form of the Map-STDP rule:

$$G_{sim}(i,j) = \mathbb{I}[j \notin m(i)] - \frac{e_i}{\tilde{s}_i} \tag{3}$$

Given a negative learning rate for the gating term, this simplified rule can be interpreted as follows: if $j \notin m(i)$, then $G_{sim}(i,j) = 1 - \bar{e}_i > 0$, so W_{ij} is decreased, discouraging cross-community connectivity. If $j \in m(i)$, then $G_{sim}(i,j) = -\bar{e}_i < 0$, so W_{ij} is increased, reinforcing intra-community connectivity. In both cases the magnitude of the update is modulated by $\bar{e}_i = e_i/\tilde{s}_i$, the current normalized exit probability of node i. This means nodes that are already well-contained within their community ($\bar{e}_i \approx 0$) receive small updates, while nodes with high community leakage

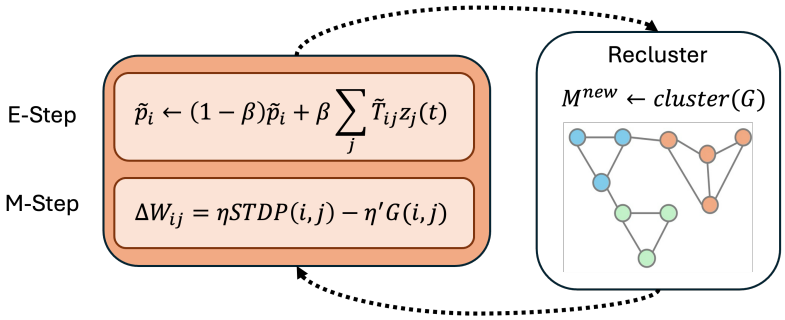

Fig. 1 The network of updates during training. The Map-STDP rule would involve an E-step where the network dynamics are run to update $\tilde{p}$, and an M-step where W is updated using the Map-STDP rule with $\tilde{p}$ held constant. The community assignments M can either be kept static or periodically updated using a community detection algorithm.

($\bar{e}_i \approx 1$) receive large corrections. The rule is self-regulating in the sense that it applies the strongest pressure where community structure is most violated.

One issue that is raised is that needing to compute e_i and $\tilde{s}_i$ for every synaptic update violates locality. However, since $W_{ij} = 0$ for non-neighbors, we can compute e_i and $\tilde{s}_i$ using only local information from the neighbors of node i:

$$e_i = \sum_{j' \in N(i)} \mathbb{I}[j' \notin m(i)] W_{ij'} \tilde{p}_{j'} \tag{4}$$

$$\tilde{s}_i = \sum_{k \in N(i)} W_{ik} \tilde{p}_k \tag{5}$$

Where $N(i)$ is the set of neighbors of node i. This means that the Map-STDP rule can be implemented in a biologically plausible manner, as it only requires information from the local neighborhood of each synapse.

4 Training

Now that we have derived the Map-STDP rule, we need to work out the mechanics of training. There are four main structures that need to be cached:

- M: the community assignment of each node. This can be stored as a vector of integers, where the i-th element is the community that node i belongs to.
- $\tilde{p}$: the stationary distribution of the random walk on the network. This can be stored as a vector of floats, where the i-th element is the stationary probability of node i.
- $N(i)$: the set of neighbors of each node. This can be stored as a list of lists, where the i-th element is a list of the neighbors of node i.
- W: the weight matrix of the network. This can be stored as a 2D array, where the element at row i and column j is the weight of the edge from node i to node j.

With this leaves us with some open questions:

- **Should M be kept static or periodically updated?** If we keep M static, then the Map-STDP rule will only reinforce the initial community structure, which may not be optimal. On the other hand, if we periodically update M using a community detection algorithm like Infomap, then the Map-STDP rule will be able to adapt to changes in the network structure and potentially find better community assignments. However, this comes at the cost of increased computational complexity due to running a community detection algorithm at each update step.
- **How do we get the SNN to update its topology?** As opposed to simply just its weights. One possibility is that we set some hyperparameter ϵ such that if $W_{ij} < \epsilon$, then we set $W_{ij} = 0$. This would allow the network to prune weak connections and potentially form more distinct communities. However, this also introduces a new hyperparameter that needs to be tuned, and it may lead to instability in the training process if not handled carefully. Another possibility is a k parameter such that all nodes have their top k strongest connections kept and the rest set to zero. This would ensure that the network maintains a certain level of connectivity while still allowing for pruning of weaker connections. However, this also introduces a new hyperparameter that needs to be tuned, and it may lead to instability if k is set too low or too high.
- **How do we initialize the SNN, and M?** One possibility is to initialize as a random Erdos-Renyi graph, with communities initialized randomly such that each node has a $1/|M|$ chance of being assigned to each community. This would provide a neutral starting point for the network to learn from, without any inherent bias towards a particular community structure. However, this also means that the network may take longer to converge to an optimal community structure, as it has to learn both the weights and the community assignments from scratch. Another possibility is to initialize with some amount of community structure, potentially using a stochastic block model. This would give the network a head start in learning the community structure, as it would already have some initial communities to work with. However, this also introduces bias towards the initial community structure, which may not be optimal for the task at hand.

It should be noted that although the Map-STDP uses the map equation as its objective function, it does not necessarily need to operate using Infomap communities. As the update rule requires any community assignment, M, we could potentially use any community detection algorithm we'd like. If the goal is to select from a constant set outcomes (e.g. classification or discrete control), then we could use methods like Infomap and SBM to produce community assignments with a constant number of communities. However, if the goal is to learn a more flexible representation of the input stimulus, then we could use methods like Louvain or Leiden that produce a variable number of communities based on the structure of the network.

This model may also benefit from a central community which broadcasts to all other communities. This central community could be linked to a receptive field that encodes the input stimulus, and it could also be linked to a readout layer that produces the output response. This would be in tune with the idea of the Global Network Workspace Theory (GNWT) [6], which posits that there is a central workspace in the brain that integrates information from different specialized modules (communities)

and broadcasts it to the rest of the brain. This could potentially allow the network to learn more complex representations of the input stimulus, as it would be able to leverage the modular structure of the network to learn more efficient representations.

References

[1] Shouval, H.Z., Wang, S.S.-H., Wittenberg, G.M.: Spike timing dependent plasticity: a consequence of more fundamental learning rules. Front. Comput. Neurosci. **4** (2010)

[2] Buesing, L., Bill, J., Nessler, B., Maass, W.: Neural dynamics as sampling: a model for stochastic computation in recurrent networks of spiking neurons. PLoS Comput. Biol. **7**(11), 1002211 (2011)

[3] Rosvall, M., Bergstrom, C.T.: Maps of random walks on complex networks reveal community structure. Proc. Natl. Acad. Sci. U. S. A. **105**(4), 1118–1123 (2008)

[4] Dayan, P., Abbott, L.F.: Information theory. In: Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems, pp. 123–150. MIT Press, Cambridge, MA (2001). https://doi.org/10.7551/mitpress/6755.001.0001

[5] Friston, K.: The free-energy principle: a unified brain theory? Nat. Rev. Neurosci. **11**(2), 127–138 (2010)

[6] Baars, B.J.: Global workspace theory of consciousness: toward a cognitive neuroscience of human experience. Prog. Brain Res. **150**, 45–53 (2005)

Appendix A Proofs

Theorem 1 *The local update rule in Equation 2 converges to the stationary distribution of the random walk on the network.*

Proof Taking the expectation of the second term in the local update rule:

$$\mathbb{E}\left[\sum_j \tilde{T}_{ij} z_j(t)\right] = \sum_j \tilde{T}_{ij}\langle z_j\rangle = \sum_j \tilde{T}_{ij}\tilde{p}_j = [\tilde{T}\tilde{p}]_i$$

Substituting back into the local update rule gives:

$$\tilde{p}_i \leftarrow (1-\beta)\tilde{p}_i + \beta[\tilde{T}\tilde{p}]_i$$

By the fixed point property of the stationary distribution $\tilde{p} = \tilde{T}\tilde{p}$, this reduces to:

$$\tilde{p}_i \leftarrow (1-\beta)\tilde{p}_i + \beta\tilde{p}_i = \tilde{p}_i$$

so $\tilde{p}$ is a fixed point of the update rule. □

Theorem 2 *The gating function $G(i,j)$ is the same as the gradient of the map equation with respect to W_{ij}, treating $\tilde{p}$ as constant with respect to W.*

Proof Start by rewriting the map equation as follows:

$$L(M) = A + B + C$$

Where:

$$A := -\sum_{m\in M} q_m \log\left(\frac{q_m}{q_\curvearrowright}\right)$$

$$B := \sum_{m\in M} q_m \log\left(\frac{q_m}{p_m + q_m}\right)$$

$$C := \sum_{m\in M}\sum_{i\in m} \tilde{p}_i \log\left(\frac{\tilde{p}_i}{p_m + q_m}\right)$$

Then, the gradient can be computed as:

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial A}{\partial W_{ij}} + \frac{\partial B}{\partial W_{ij}} + \frac{\partial C}{\partial W_{ij}}$$

Starting with $\partial A/\partial W_{ij}$:

$$\begin{aligned}\frac{\partial A}{\partial W_{ij}} &= -\sum_{m\in M}\left[\frac{\partial q_m}{\partial W_{ij}}\log\left(\frac{q_m}{q_\curvearrowright}\right) + q_m\frac{\partial}{\partial W_{ij}}\log\left(\frac{q_m}{q_\curvearrowright}\right)\right] \\ &= -\sum_{m\in M}\left[\frac{\partial q_m}{\partial W_{ij}}\log\left(\frac{q_m}{q_\curvearrowright}\right) + \frac{\partial q_m}{\partial W_{ij}} - \frac{q_m}{q_\curvearrowright}\frac{\partial q_\curvearrowright}{\partial W_{ij}}\right]\end{aligned}$$

Now, we can simplify this even further by computing $\partial q_\curvearrowright/\partial W_{ij}$:

$$\frac{\partial q_\curvearrowright}{\partial W_{ij}} = \sum_{m\in M}\frac{\partial q_m}{\partial W_{ij}} = \frac{\partial q_{m(i)}}{\partial W_{ij}}$$

Where $m(i)$ is the community that node i belongs to. This is because q_m only depends on W_{ij} if i belongs to community m. Now if we expand the second and third terms of the gradient, we can see that they cancel each other out:

$$\begin{aligned}\frac{\partial A}{\partial W_{ij}} &= -\sum_{m\in M}\left[\frac{\partial q_m}{\partial W_{ij}}\log\left(\frac{q_m}{q_\curvearrowright}\right) + \frac{\partial q_m}{\partial W_{ij}} - \frac{q_m}{q_\curvearrowright}\frac{\partial q_{m(i)}}{\partial W_{ij}}\right] \\ &= -\sum_{m\in M}\frac{\partial q_m}{\partial W_{ij}}\log\left(\frac{q_m}{q_\curvearrowright}\right) - \sum_{m\in M}\frac{\partial q_m}{\partial W_{ij}} + \sum_{m\in M}\frac{q_m}{q_\curvearrowright}\frac{\partial q_m}{\partial W_{ij}} \\ &= -\left[\sum_{m\in M}\frac{\partial q_m}{\partial W_{ij}}\log\left(\frac{q_m}{q_\curvearrowright}\right)\right] - \frac{\partial q_{m(i)}}{\partial W_{ij}} + \frac{\partial q_{m(i)}}{\partial W_{ij}}\cdot\frac{1}{q_\curvearrowright}\sum_{m\in M} q_m \\ &= -\left[\sum_{m\in M}\frac{\partial q_m}{\partial W_{ij}}\log\left(\frac{q_m}{q_\curvearrowright}\right)\right] - \frac{\partial q_{m(i)}}{\partial W_{ij}} + \frac{\partial q_{m(i)}}{\partial W_{ij}}\cdot\frac{1}{\sum_{m\in M} q_m}\sum_{m\in M} q_m\end{aligned}$$

Finally, this simplifies to:

$$\frac{\partial A}{\partial W_{ij}} = -\sum_{m \in M} \frac{\partial q_m}{\partial W_{ij}} \log\left(\frac{q_m}{q_\curvearrowright}\right)$$

Now for $\partial B / \partial W_{ij}$:

$$\frac{\partial B}{\partial W_{ij}} = \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log\left(\frac{q_m}{p_m + q_m}\right) + q_m \frac{\partial}{\partial W_{ij}} \log\left(\frac{q_m}{p_m + q_m}\right)\right]$$

$$= \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log\left(\frac{q_m}{p_m + q_m}\right) + q_m \left(\frac{1}{q_m}\frac{\partial q_m}{\partial W_{ij}} - \frac{1}{p_m + q_m}\left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}}\right)\right)\right]$$

$$= \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log\left(\frac{q_m}{p_m + q_m}\right) + \frac{\partial q_m}{\partial W_{ij}} - \frac{q_m}{p_m + q_m}\left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}}\right)\right]$$

And finally for $\partial C / \partial W_{ij}$:

$$\frac{\partial C}{\partial W_{ij}} = \sum_{m \in M} \sum_{k \in m} \left[\frac{\partial \tilde{p}_k}{\partial W_{ij}} \log\left(\frac{\tilde{p}_k}{p_m + q_m}\right) + \tilde{p}_k \frac{\partial}{\partial W_{ij}} \log\left(\frac{\tilde{p}_k}{p_m + q_m}\right)\right]$$

$$= \sum_{m \in M} \sum_{k \in m} \left[\frac{\partial \tilde{p}_k}{\partial W_{ij}} \log\left(\frac{\tilde{p}_k}{p_m + q_m}\right) + \tilde{p}_k \left(\frac{1}{\tilde{p}_k}\frac{\partial \tilde{p}_k}{\partial W_{ij}} - \frac{1}{p_m + q_m}\left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}}\right)\right)\right]$$

$$= \sum_{m \in M} \sum_{k \in m} \left[\frac{\partial \tilde{p}_k}{\partial W_{ij}} \log\left(\frac{\tilde{p}_k}{p_m + q_m}\right) + \frac{\partial \tilde{p}_k}{\partial W_{ij}} - \frac{\tilde{p}_k}{p_m + q_m}\left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}}\right)\right]$$

Since we are treating $\tilde{p}$ as a constant with respect to W_{ij}, the $\partial \tilde{p}_k / \partial W_{ij}$ terms can be removed from the gradient, which gives us the final form of the gradient as:

$$\frac{\partial C}{\partial W_{ij}} = \sum_{m \in M} \sum_{k \in m} \left[-\frac{\tilde{p}_k}{p_m + q_m}\left(\frac{\partial p_m}{\partial W_{ij}} + \frac{\partial q_m}{\partial W_{ij}}\right)\right]$$

Similarly, we can adjust all terms and remove all $\partial \tilde{p}_i / \partial W_{ij}$ terms, which gives us the following:

$$\frac{\partial B}{\partial W_{ij}} = \sum_{m \in M} \left[\frac{\partial q_m}{\partial W_{ij}} \log\left(\frac{q_m}{p_m + q_m}\right) + \frac{\partial q_m}{\partial W_{ij}} - \frac{q_m}{p_m + q_m}\left(\frac{\partial q_m}{\partial W_{ij}}\right)\right]$$

$$= \sum_{m \in M} \frac{\partial q_m}{\partial W_{ij}} \left[\log\left(\frac{q_m}{p_m + q_m}\right) + 1 - \frac{q_m}{p_m + q_m}\right]$$

$$\frac{\partial C}{\partial W_{ij}} = \sum_{m \in M} \sum_{k \in m} \left[-\frac{\tilde{p}_k}{p_m + q_m}\left(\frac{\partial q_m}{\partial W_{ij}}\right)\right]$$

$$= -\sum_{m \in M} \frac{\partial q_m}{\partial W_{ij}} \left[\sum_{k \in m} \frac{\tilde{p}_k}{p_m + q_m}\right]$$

Combining these terms, all three share a factor of $\partial q_m / \partial W_{ij}$. Grouping and using $\sum_{k \in m} \tilde{p}_k = p_m$:

$$\frac{\partial L}{\partial W_{ij}} = \sum_{m \in M} \frac{\partial q_m}{\partial W_{ij}} \left[-\log\frac{q_m}{q_\curvearrowright} + \log\frac{q_m}{p_m + q_m} + 1 - \frac{q_m}{p_m + q_m} - \frac{p_m}{p_m + q_m}\right]$$

The last two terms cancel since $1 - \frac{q_m}{p_m + q_m} - \frac{p_m}{p_m + q_m} = 0$, and the logarithms combine as $-\log\frac{q_m}{q_\curvearrowright} + \log\frac{q_m}{p_m + q_m} = \log\frac{q_\curvearrowright}{p_m + q_m}$, giving the final gradient:

$$\frac{\partial L}{\partial W_{ij}} = \sum_{m \in M} \frac{\partial q_m}{\partial W_{ij}} \log \frac{q_{\curvearrowright}}{p_m + q_m}$$

We can simplify the gradient even further as $\partial q_m / \partial W_{ij}$ only depends on the community that node i belongs to. Therefore, we can rewrite the gradient as:

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial q_{m(i)}}{\partial W_{ij}} \log \frac{q_{\curvearrowright}}{p_{m(i)} + q_{m(i)}}$$

Now, we need to compute $\partial q_{m(i)} / \partial W_{ij}$. We can rewrite $q_{m(i)}$ as:

$$\begin{aligned}
q_{m(i)} &= \sum_{i' \in m(i)} \sum_{j' \notin m(i)} \tilde{T}_{i'j'} \tilde{p}_{i'} \\
&= \sum_{i' \in m(i)} \sum_{j' \notin m(i)} \frac{\tilde{W}_{i'j'}}{\tilde{s}_{i'}} \tilde{p}_{i'} \\
&= \sum_{i' \in m(i)} \sum_{j' \notin m(i)} \frac{W_{i'j'} \tilde{p}_{j'}}{\sum_k W_{i'k} \tilde{p}_k} \tilde{p}_{i'} \\
&= \sum_{i' \in m(i)} \tilde{p}_{i'} \sum_{j' \notin m(i)} \frac{W_{i'j'} \tilde{p}_{j'}}{\sum_k W_{i'k} \tilde{p}_k}
\end{aligned}$$

The terms in $q_{m(i)}$ for all other $i' \neq i$ do not depend on W_{ij}, so they can be removed from the derivative, giving us:

$$\frac{\partial q_{m(i)}}{\partial W_{ij}} = \tilde{p}_i \frac{\partial}{\partial W_{ij}} \left[\frac{\sum_{j' \notin m(i)} W_{ij'} \tilde{p}_{j'}}{\sum_k W_{ik} \tilde{p}_k} \right]$$

This can be computed using the quotient rule:

$$\begin{aligned}
\frac{\partial q_{m(i)}}{\partial W_{ij}} &= \tilde{p}_i \frac{\left(\sum_k W_{ik} \tilde{p}_k\right) \tilde{p}_j \mathbb{I}[j \notin m(i)] - \left(\sum_{j' \notin m(i)} W_{ij'} \tilde{p}_{j'}\right) \tilde{p}_j}{\left(\sum_k W_{ik} \tilde{p}_k\right)^2} \\
&= \tilde{p}_i \tilde{p}_j \frac{\mathbb{I}[j \notin m(i)] \left(\sum_k W_{ik} \tilde{p}_k\right) - \sum_{j' \notin m(i)} W_{ij'} \tilde{p}_{j'}}{\left(\sum_k W_{ik} \tilde{p}_k\right)^2} \\
&= \tilde{p}_i \tilde{p}_j \frac{\mathbb{I}[j \notin m(i)] \tilde{s}_i - e_i}{\tilde{s}_i^2}
\end{aligned}$$

Where $e_i = \sum_{j' \notin m(i)} W_{ij'} \tilde{p}_{j'}$ is the total exit flow from node i. Substituting this back into the gradient gives us the final form of $G(i, j)$:

$$\frac{\partial L}{\partial W_{ij}} = \tilde{p}_i \tilde{p}_j \frac{\mathbb{I}[j \notin m(i)] \tilde{s}_i - e_i}{\tilde{s}_i^2} \log \frac{q_{\curvearrowright}}{p_{m(i)} + q_{m(i)}}$$

□